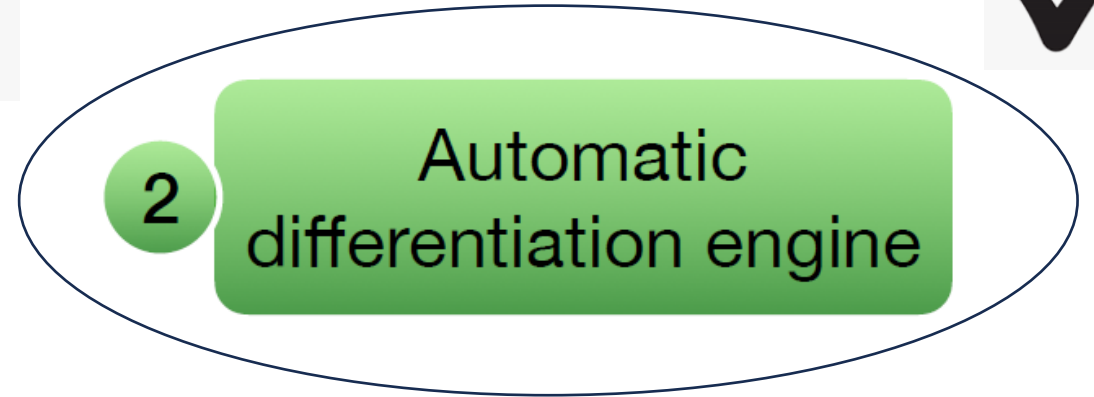
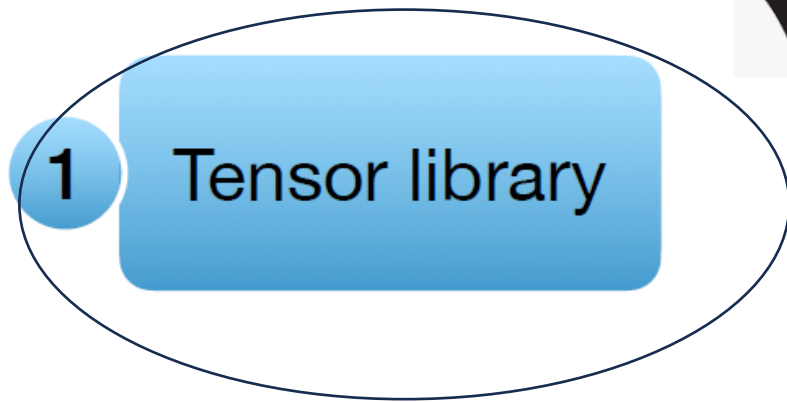
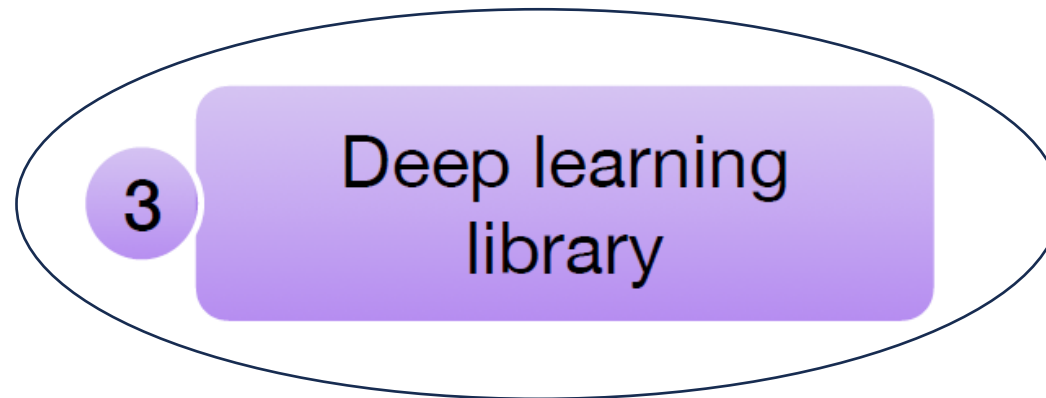


L4 Linear Regression



What is PyTorch?



Simple Linear Regression Basics

- Linear regression is a linear model, a model that assumes a linear relationship between the input variables (x) and the single output variable (y).
- More specifically, that y can be calculated from a linear combination of the input variables (x).
- When there is a single input variable (x), the method is referred to as **simple linear regression**.
- When there are multiple input variables, **multiple linear regression**.

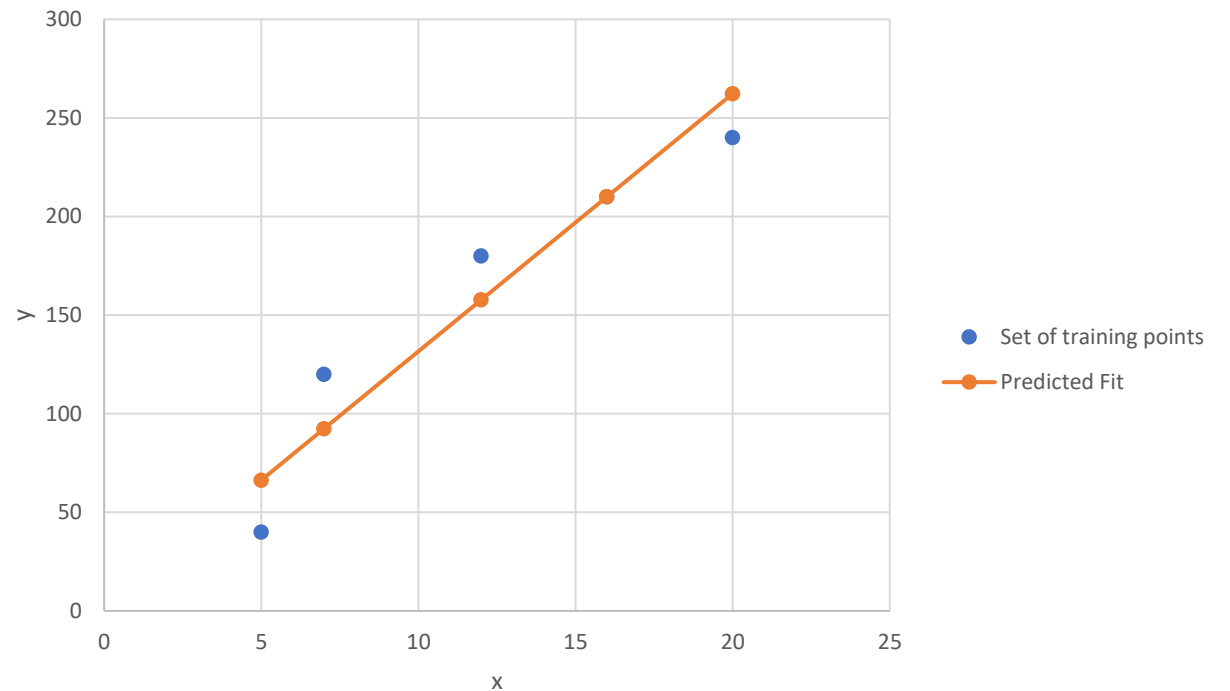
Simple Linear Regression

- Allows us to understand **relationship** between two **continuous variables**
- Example
 - x: independent variable
 - weight
 - y: dependent variable
 - height
- $y=wx+b$

X	Y
5	40
7	120
12	180
16	210
20	240

Simple Linear Regression

- Aim of Linear Regression
- Minimize the distance between the points and the line ($y=\alpha x+\beta$)
- Adjusting
 - Coefficient: w
 - Bias/intercept: b
- Learnable parameters: w, b



Simple Linear Regression

- MSE Loss: Mean Squared Error

- $MSE = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$

- $\hat{y}$: prediction
- y : true value

Simple Linear Regression

- Review of Gradient Descent

- Simplified equation

- $\theta = \theta - \eta \cdot \nabla_{\theta}$

- θ : parameters (our variables)

- η : learning rate (how fast we want to learn)

- ∇_{θ} : parameters' gradients

- Even simpler equation

- `parameters = parameters - learning_rate * parameters_gradients`

Building a Linear Regression Model with PyTorch

```
# Create the tensors x and y. They are the training
# examples in the dataset for the linear regression (Labelled Dataset)
x = torch.tensor([5.0, 7.0, 12.0, 16.0, 20.0])
y = torch.tensor([40.0, 120.0, 180.0, 210.0, 240.0])

# The learning rate is set to alpha = 0.001
learning_rate = torch.tensor(0.001)
```


Building a Linear Regression Model with PyTorch

- # The parameters to be learnt w , and b in the
 - # prediction $y_p = wx + b$
- ```
b = torch.rand([1], requires_grad=True)
w = torch.rand([1], requires_grad=True)
print("The parameters are {}, and {}".format(w, b))
```

# Simple Linear Regression Basics

- Use the simple linear equation  $y=wx+b$  as an example
- The parameters of the equation are
  - slope  $w$
  - intercept  $b$
- 2 ways
  1. Building simple regression
  2. Building a Custom Linear Class

# Simple Linear Regression

- one-dimensional linear regression having only two parameters.
- Create the linear expression  $Y=3x+1$
- Define the parameters  $w$  and  $b$  as tensors in PyTorch.
- set the `requires_grad` parameter to `True`, indicating that our model has to learn these parameters:

```
import torch

defining the parameters 'w' and 'b'
w = torch.tensor(3.0, requires_grad = True)
b = torch.tensor(1.0, requires_grad = True)
```

# Simple Linear Regression

- Write a function to make predictions for  $y$  at any given value of  $x$
- In PyTorch prediction step is called forward step.

# function of the linear equation for making predictions

```
def forward(x):
```

```
 y_pred = w * x + b
```

```
 return y_pred
```

# predict  $y_{\text{pred}}$  at  $x = 2$

```
x = torch.tensor([[2.0]])
```

```
y_pred = forward(x)
```

```
print("prediction of y at ' $x = 2$ ' is: ", y_pred)
```

# Simple Linear Regression— complete code

```
import torch

defining the parameters 'w' and 'b'
w = torch.tensor(3.0, requires_grad = True)
b = torch.tensor(1.0, requires_grad = True)

function of the linear equation for making predictions
def forward(x):
 y_pred = w * x + b
 return y_pred

predict y_pred at x = 2
x = torch.tensor([[2.0]])
y_pred = forward(x)
print("prediction of y at 'x = 2' is: ", y_pred)

making predictions at multiple values of x
x = torch.tensor([[3.0], [4.0]])
y_pred = forward(x)
print("prediction of y at 'x = 3 & 4' is: ", y_pred)
```

# Simple Linear Regression

prediction of y at 'x = 2' is: `tensor([[7.]], grad_fn=<AddBackward0>)`

prediction of y at 'x = 3 & 4' is: `tensor([[10.],  
[13.]], grad_fn=<AddBackward0>)`

# Train a simple linear regression model

- In order to train a linear regression model, we need to define a cost function and an optimizer.
- The **cost function** is used to measure how well our model fits the data,
- while the **optimizer** decides which direction to move in order to improve this fit.
- Training the model and updating the parameters after going through a single iteration of training data is known as one **epoch**
- We should train the model for **several epochs** so that weights and biases can learn the linear relationship between the input features and output labels.
- So far, used simple predictions with only a linear regression forward pass
- Now, train a linear regression model and update its learning parameters

# Building a Linear Regression Model with PyTorch

```
for epochs in range(2):
 #Compute the average loss for the training samples
 loss = 0.0
 #Accumulate the loss for all the samples
 for j in range(len(x)):
 a = w * x[j]
 y_p = a + b
 print("y_p=", y_p)
 loss += (y_p-y[j]) ** 2
 #loss += (y[j]-y_p)**2
 print("loss=", loss)
 #Find the average loss
 loss = loss / len(x)
```



# Building a Linear Regression Model with PyTorch

```
#Compute the gradients using backward
loss.backward()

Without modifying the gradient in this block| perform the operation
with torch.no_grad():
 # Update the weight based on gradient descent
 # equivalently one may write w1.copy_(w1 - learning_rate * w1.grad)
 w -= learning_rate * w.grad
 b -= learning_rate * b.grad

#reset the gradients for next epoch
w.grad.zero_()
b.grad.zero_()

#Display the parameters and loss
print("The parameters are w={}, b={}, and loss={}".format(w, b, loss.item()))

end of for loop
```

Note: Resetting gradients to zero because PyTorch accumulates gradients

# Problem 1

- Find the value of  $w.grad$  and  $b.grad$  using analytical solution for the given linear regression problem.
- Initial value of  $w = b = 1$
- MSE Loss

| x | y  |
|---|----|
| 2 | 20 |
| 4 | 40 |

## Problem 1– Verify loss, w.grad, b.grad, new w, new b

```
x = torch.tensor([2, 4])
y = torch.tensor([20, 40])
b = torch.tensor([1.0], requires_grad=True)
w = torch.tensor([1.0], requires_grad=True)
learning_rate = torch.tensor(0.001)
loss = 0.0
for j in range(len(x)):
 a = w * x[j]
 y_p = a + b
 loss += (y_p - y[j]) ** 2
loss = loss / len(x)
print("avg loss=", loss)
```

```
loss.backward()
print("w.grad=", w.grad)
with torch.no_grad():
 w -= learning_rate * w.grad
 b -= learning_rate * b.grad
w.grad.zero_()
b.grad.zero_()
```

## Sample data – Verify loss, w.grad, b.grad, new w, new b

```
x = torch.tensor([2, 4])
y = torch.tensor([20, 40])
b = torch.tensor([1.0], requires_grad=True)
w = torch.tensor([1.0], requires_grad=True)
learning_rate = torch.tensor(0.001)
loss = 0.0
for j in range(len(x)):
 a = w * x[j]
 y_p = a + b
 loss += (y_p - y[j]) ** 2
loss = loss / len(x)
print("avg loss=", loss)
```

```
loss.backward()
print("w.grad=", w.grad)
with torch.no_grad():
 w -= learning_rate * w.grad
 b -= learning_rate * b.grad
w.grad.zero_()
b.grad.zero_()
```

```
w.grad= tensor([-174.])
b.grad= tensor([-52.])
The parameters are
w=1.1740000247955322,
b=1.0520000457763672, and
loss=757.0
```

# Complete code with epoch

```
import torch
from matplotlib import pyplot as plt
Create the tensors x and y. They are the training
examples in the dataset for the linear regression
x = torch.tensor([2, 4])
y = torch.tensor([20, 40])
b = torch.tensor([1.0], requires_grad=True)
w = torch.tensor([1.0], requires_grad=True)
print("The parameters are {}, and {}".format(w, b))
The learning rate is set to alpha = 0.001
learning_rate = torch.tensor(0.001)
#The list of loss values for the plotting purpose
loss_list = []
```

```
for epochs in range(10):
 #Compute the average loss for the training samples
 loss = 0.0
 #Accumulate the loss for all the samples
 for j in range(len(x)):
 a = w * x[j]
 y_p = a + b
 loss += (y_p-y[j]) ** 2
 #loss += (y[j]-y_p)**2
 #Find the average loss
 loss = loss / len(x)
 print("avg loss=", loss)
 #Add the loss to a list for the plotting purpose
 loss_list.append(loss.item())
```

Additional question: Try `w.grad` if `loss=(y[j]-y_p)**2`

# Complete code with epoch

```
#Compute the gradients using backward
dl/dw and dl/db
loss.backward()
print("w.grad=", w.grad)
Without modifying the gradient in this block
perform the operation
with torch.no_grad():
 # Update the weight based on gradient descent

 w -= learning_rate * w.grad
 b -= learning_rate * b.grad

w.grad.zero_()
b.grad.zero_()

#w.grad = None
#b.grad = None

print("The parameters are w={}, b={}, and
loss={}".format(w.item(), b.item(), loss.item()))

#Display the plot – End of outer epoch for loop
plt.plot(loss_list)
plt.show()
print(loss.item(), loss.data)
```

# Revised Version of the Implementation

```
Create the tensors x and y. They are the training
examples in the dataset for the linear regression (Labelled Dataset)
x = torch.tensor([5.0, 7.0, 12.0, 16.0, 20.0])
y = torch.tensor([40.0, 120.0, 180.0, 210.0, 240.0])

learning_rate = torch.tensor(0.001)
```

# Revised Version of the Implementation

```
class RegressionModel:
 def __init__(self):
 self.w = torch.rand([1], requires_grad=True)
 self.b = torch.rand([1], requires_grad=True)

 def forward(self, x):
 return self.w * x + self.b

 def update(self):
 # Update the weight based on gradient descent
 # equivalently one may write w1.copy_(w1 - learning_rate * w1.grad)
 self.w -= learning_rate * self.w.grad
 self.b -= learning_rate * self.b.grad

 def reset_grad(self):
 self.w.grad.zero_()
 self.b.grad.zero_()
```



# Revised Version of the Implementation

```
def criterion(yj, y_p):
 return (yj - y_p)**2
```

```
model = RegressionModel()

#The list of loss values for the plotting purpose
loss_list = []

Run the training loop for N epochs
for epochs in range(100):
 loss = 0.0
 #Accumulate the loss for all the samples
 for j in range(len(x)):
 y_p = model.forward(x[j])
 loss += criterion(y[j], y_p)

 #Find the average loss
 loss = loss / len(x)
 #Add the loss to a list for the plotting purpose
 loss_list.append(loss.item())
```

# Revised Version of the Implementation

```
#Compute the gradients using backward dl/dw and dl/db
```

```
loss.backward()
```

```
Without modifying the gradient in this block
```

```
perform the operation
```

```
with torch.no_grad():
```

```
 model.update()
```

```
#reset the gradients for next epoch
```

```
model.reset_grad()
```

```
#w.grad = None
```

```
#b.grad = None
```

```
prev_loss = loss
```

```
#Display the parameters and loss
```

```
print("The parameters are w={}, b={}, and loss={}".format(model.w, model.b, loss.item()))
```

```
#Display the plot
```

```
plt.plot(loss_list)
```

```
plt.show()
```

# Simple Linear Regression Basics -Building a Custom Linear Class

- PyTorch offers the possibility to build custom linear class.
- This method will be used for building complex models.
- Start by importing the nn module from PyTorch in order to build a custom linear class.
- `from torch import nn`
- Custom modules in PyTorch are classes derived from `nn.Module`.
- Build a class for simple linear regression and name it as `Linear_Regression`. make it a child class of the `nn.Module`.
- So all the methods and attributes will be inherited into this class.
- In the object constructor, declare the input and output parameters.
- Also, create a super constructor to call linear class from the `nn.Module`.
- In order to generate prediction from the input samples, define a `forward function` in the class.
- Once we have created the dataset, define the model architecture. The code is shown next

# Simple Linear Regression Basics -Building a Custom Linear Class

```
class Linear_Regression(nn.Module):
 def __init__(self, input_sample, output_sample):
 # Inheriting properties from the parent calss
 super(Linear_Regression, self).__init__()
 self.linear = nn.Linear(input_sample, output_sample)

 # define function to make predictions
 def forward(self, x):
 output = self.linear(x)
 return output
```

# Linear Regression using PyTorch built-ins

- PyTorch provides the modules and classes `torch.nn` Dataset , and `DataLoader` to create and train neural networks.
- Dataset : PyTorch's `TensorDataset` is a Dataset wrapping tensors. By defining a length and way of indexing, this also gives us a way to iterate, index, and slice along the first dimension of a tensor. This will make it easier to access both the independent and dependent variables in the same line as we train.
- DataLoader: Pytorch's `DataLoader` is responsible for managing batches. We can create a `DataLoader` from any Dataset. `DataLoader` makes it easier to iterate over batches.
- Model: Instead of initializing the weights & biases manually, we can define the model using `torch.nn.Linear`
- Optimizer: We can make use of the built Stochastic Gradient Descent optimizer ( `torch.optim.SGD` )

# Optim Module

- The Optim module in PyTorch has pre-written codes for most of the optimizers that are used while building a neural network.
- We just have to import them and then they can be used to build models.
- Ex: to get SDG Optimizer supported in PyTorch

```
importing the optim module
```

```
from torch import optim
```

```
SGD = optim.SGD(model.parameters(), lr=learning_rate)
```

# PyTorch Deep Learning Model Life-Cycle

- Life-cycle for a deep learning model and the PyTorch API that we can use to define models
- The five steps in the life-cycle are as follows:
  1. Prepare the Data
  2. Define the Model
  3. Train the Model
  4. Evaluate the Model
  5. Make Predictions

# Step 1: Prepare the Data

- The first step is to load and prepare our data.
- Neural network models require numerical input data and numerical output data.
- PyTorch provides the `Dataset` class that we can extend and customize to load our dataset.
- the constructor of our dataset object can load our data file (e.g. a CSV file).
- We can then override
  - `__len__()` function that can be used to get the length of the dataset (number of rows or samples), and
  - `__getitem__()` function that is used to get a specific sample by index.
- Note: When loading our dataset, we can also perform any required transforms, such as scaling or encoding.
- A skeleton of a custom Dataset class is provided below.



# Linear Regression Implementation: Fully PyTorch way

```
class MyDataset(Dataset):
 def __init__(self, X, Y):
 self.X = X
 self.Y = Y

 def __len__(self):
 return len(self.X)

 def __getitem__(self, idx):
 return self.X[idx].to(device), self.Y[idx].to(device)
```

# Step 1: Prepare the Data

#dataset definition

```
class MyDataset(Dataset):
 # load the dataset
 def __init__(self, path):
 # store the inputs and outputs
 self.X = ...
 self.y = ...

 # number of rows in the dataset
 def __len__(self):
 return len(self.X)

 # get a row at an index
 def __getitem__(self, idx):
 return [self.X[idx], self.y[idx]]
```

# Data read - DataLoader

```
from torch.utils.data import Dataset, DataLoader
import torch
```

```
x = torch.tensor(
 [12.4, 14.3, 14.5, 14.9, 16.1, 16.9, 16.5, 15.4, 17.0, 17.9, 18.8, 20.3, 22.4, 19.4, 15.5, 16.7,
 17.3, 18.4, 19.2,
 17.4, 19.5, 19.7, 21.2])
```

```
y = torch.tensor(
 [11.2, 12.5, 12.7, 13.1, 14.1, 14.8, 14.4, 13.4, 14.9, 15.6, 16.4, 17.7, 19.6, 16.9, 14.0, 14.6,
 15.1, 16.1, 16.8,
 15.2, 17.0, 17.2, 18.6])
```

```
#23 values
```

```
#Find if CUDA is available to load the model and device on to the available device CPU/GPU
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

# Data read - DataLoader

```
class MyDataset(Dataset):
 def __init__(self, X, Y):
 self.X = X
 self.Y = Y

 def __len__(self):
 return len(self.X)

 def __getitem__(self, idx):
 return self.X[idx].to(device), self.Y[idx].to(device)

dataset = MyDataset(x,y)
dataloader = DataLoader(dataset, batch_size=4, shuffle=True)

for data in iter(dataloader):
 print(data)
```

# Data read - DataLoader

```
x = torch.tensor([12.4, 14.3, 14.5, 14.9, 16.1, 16.9, 16.5, 15.4, 17.0, 17.9, 18.8, 20.3, 22.4, 19.4, 15.5, 16.7, 17.3, 18.4, 19.2, 17.4, 19.5, 19.7, 21.2])
```

```
y = torch.tensor([11.2, 12.5, 12.7, 13.1, 14.1, 14.8, 14.4, 13.4, 14.9, 15.6, 16.4, 17.7, 19.6, 16.9, 14.0, 14.6, 15.1, 16.1, 16.8, 15.2, 17.0, 17.2, 18.6])
```

```
#Total 23 = 4*5+3
```

```
dataloader = DataLoader(dataset, batch_size=4, shuffle=True)
```

```
[tensor([19.5000, 14.9000, 20.3000, 14.5000]), tensor([17.0000, 13.1000, 17.7000, 12.7000])]
```

```
[tensor([17.4000, 15.4000, 19.7000, 14.3000]), tensor([15.2000, 13.4000, 17.2000, 12.5000])]
```

```
[tensor([17.0000, 22.4000, 19.2000, 16.9000]), tensor([14.9000, 19.6000, 16.8000, 14.8000])]
```

```
[tensor([16.7000, 17.9000, 18.8000, 18.4000]), tensor([14.6000, 15.6000, 16.4000, 16.1000])]
```

```
[tensor([12.4000, 16.5000, 21.2000, 19.4000]), tensor([11.2000, 14.4000, 18.6000, 16.9000])]
```

```
[tensor([17.3000, 16.1000, 15.5000]), tensor([15.1000, 14.1000, 14.0000])] #3 values
```

```
dataloader = DataLoader(dataset, batch_size=4, shuffle=False)
```

```
[tensor([12.4000, 14.3000, 14.5000, 14.9000]), tensor([11.2000, 12.5000, 12.7000, 13.1000])]
```

```
[tensor([16.1000, 16.9000, 16.5000, 15.4000]), tensor([14.1000, 14.8000, 14.4000, 13.4000])]
```

```
[tensor([17.0000, 17.9000, 18.8000, 20.3000]), tensor([14.9000, 15.6000, 16.4000, 17.7000])]
```

```
[tensor([22.4000, 19.4000, 15.5000, 16.7000]), tensor([19.6000, 16.9000, 14.0000, 14.6000])]
```

```
[tensor([17.3000, 18.4000, 19.2000, 17.4000]), tensor([15.1000, 16.1000, 16.8000, 15.2000])]
```

```
[tensor([19.5000, 19.7000, 21.2000]), tensor([17.0000, 17.2000, 18.6000])] #3 values
```

# Step 1: Prepare the Data

- Once loaded, PyTorch provides the `DataLoader` class to navigate a `Dataset` instance during the training and evaluation of our model.
- A `DataLoader` instance can be created for the training dataset, test dataset, and even a validation dataset.
- The `random_split()` function can be used to split a dataset into train and test sets.
- Once split, a selection of rows from the `Dataset` can be provided to a `DataLoader`, along with the batch size and whether the data should be shuffled every epoch.
- Ex: we can define a `DataLoader` by passing in a selected sample of rows in the dataset.

# Step 1: Prepare the Data

...

# create the dataset

dataset = CSVDataset(...)

# select rows from the dataset

train, test = random\_split(dataset, [[...], [...]])

# create a data loader for train and test sets

train\_dl = DataLoader(train, batch\_size=32, shuffle=True)

test\_dl = DataLoader(test, batch\_size=1024, shuffle=False)

Once defined, a DataLoader can be enumerated, yielding one batch worth of samples each iteration.

...

# train the model

for i, (inputs, targets) in enumerate(train\_dl):

...

# Step 2: Define the Model

- The next step is to define a model.
- Defining a model in PyTorch involves defining a class that extends the Module class.
- The constructor of our class defines the layers of the model and
- the forward() function is the override that defines how to forward propagate input through the defined layers of the model.
- Many layers are available, such as Linear for fully connected layers, Conv2d for convolutional layers, and MaxPool2d for pooling layers.
- Activation functions can also be defined as layers, such as ReLU, Softmax, and Sigmoid.
- Ex: a simple MLP model with one layer



# Step 2: Define the Model

```
model definition
class MLP(nn.Module):
 # define model elements
 def __init__(self, n_inputs):
 super(MLP, self).__init__()
 self.layer = Linear(n_inputs, 1)
 self.activation = Sigmoid()

 # forward propagate input
 def forward(self, X):
 X = self.layer(X)
 X = self.activation(X)
 return X
```

The super call delegates the function call to its parent class, which is nn.Module here.

This is needed to initialize the nn.Module in a proper manner

A Multilayer Perceptron model, or MLP is a standard fully connected neural network model.

It is comprised of layers of nodes where each node is connected to all outputs from the previous layer and the output of each node is connected to all inputs for nodes in the next layer.

An MLP is a model with one or more fully connected layers. This model is appropriate for tabular data, with one column for each variable and one row for each variable.

# Step 3: Train the Model

- The training process requires that we define a loss function and an optimization algorithm.
- Common loss functions include the following:
  - BCELoss: Binary cross-entropy loss for binary classification.
  - CrossEntropyLoss: Categorical cross-entropy loss for multi-class classification.
  - MSELoss: Mean squared loss for regression.
- Stochastic gradient descent is used for optimization, and the standard algorithm is provided by the SGD class
- Ex: other versions of the algorithm are available, such as Adam.
- # define the optimization

```
criterion = MSELoss()
```

```
optimizer = SGD(model.parameters(), lr=0.001)
```

# Step 3: Train the Model

- Training the model involves enumerating the DataLoader for the training dataset.
- First, a loop is required for the number of training epochs. Then an inner loop is required for the mini-batches for stochastic gradient descent.

...

```
enumerate epochs
```

```
for epoch in range(100):
```

```
 # enumerate mini batches
```

```
 for i, (inputs, targets) in enumerate(train_dl):
```

```
 ...
```

# Step 3: Train the Model

Each update to the model involves the same general pattern comprised of:

- Clearing the last error gradient.
- A forward pass of the input through the model.
- Calculating the loss for the model output.
- Backpropagating the error through the model.
- Update the model in an effort to reduce loss.

## Step 3: Train the Model

...

# clear the gradients

optimizer.zero\_grad()

# compute the model output

yhat = model(inputs)

# calculate loss

loss = criterion(yhat, targets)

# credit assignment

loss.backward()

# update model weights

optimizer.step()

## Step 4: Evaluate the model

- Once the model is fit, it can be evaluated on the test dataset.
  - This can be achieved by using the DataLoader for the test dataset and
  - collecting the predictions for the test set, then
  - comparing the predictions to the expected values of the test set and calculating a performance metric.

...

```
for i, (inputs, targets) in enumerate(test_dl):
```

```
 # evaluate the model on the test set
```

```
 yhat = model(inputs)
```

```
 ...
```

# Step 5: Make predictions

- A fit model can be used to make a prediction on new data.
- Ex: we have a single image or a single row of data and want to make a prediction.
- This requires that we wrap the data in a PyTorch Tensor data structure.
- A Tensor is the PyTorch array for holding data. It also allows to perform the automatic differentiation tasks in the model graph, like calling `backward()` when training the model.
- The prediction too will be a Tensor, although we can retrieve the NumPy array by detaching the Tensor from the automatic differentiation graph and calling the NumPy function.

# Step 5: Make predictions

...

# convert row to data

```
row = Variable(Tensor([row]).float())
```

# make prediction

```
yhat = model(row)
```

# retrieve numpy array

```
yhat = yhat.detach().numpy()
```



# Building a Linear Class

```
import torch
from torch import nn
torch.manual_seed(42)
class Linear_Regression(nn.Module):
 def __init__(self, input_sample, output_sample):
 # Inheriting properties from the parent class
 super(Linear_Regression, self).__init__()
 self.linear = nn.Linear(input_sample, output_sample)
 # define function to make predictions
 def forward(self, x):
 output = self.linear(x)
 return output
```

```
model = Linear_Regression(input_sample=1,
 output_sample=1)
print("printing the model parameters: ",
 list(model.parameters()))
x = torch.tensor([[2.0]])
y_pred = model(x)
print("getting the prediction for x: ", y_pred)
x = torch.tensor([[3.0], [4.0]])
y_pred = model(x)
print("prediction of y at 'x = 3 & 4' is: ", y_pred)
```

# Building a Linear Class - Output

printing the model parameters: [Parameter containing:  
tensor([[0.7645]], requires\_grad=True), Parameter containing:  
tensor([0.8300], requires\_grad=True)]

getting the prediction for x: tensor([[2.3591]],  
grad\_fn=<AddmmBackward0>)

prediction of y at 'x = 3 & 4' is: tensor([[3.1236],  
[3.8882]], grad\_fn=<AddmmBackward0>)

# Multilinear regression model

- The multilinear regression model is a supervised learning algorithm that can be used to predict the target variable  $y$  given multiple input variables  $x$ .
- It is a linear regression problem where more than one input variables  $x$  or features are used to predict the target variable  $y$ .
- A typical use case of this algorithm is predicting the price of a house given its size, number of rooms, and age.